

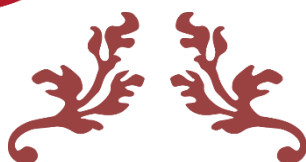
UNIVERSIDAD NACIONAL DEL SANTA

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA



UNS
UNIVERSIDAD
NACIONAL DEL SANTA



“LABORATORIO: REPRESENTACIONES CROMOSÓMICAS”



AUTORES:

- PORTAL IBAÑEZ, Anderzon Dica

CURSO:

- Algoritmos Evolutivos

ASESOR:

- LOPEZ HEREDIA, Johan Max Alexa

NUEVO CHIMBOTE –PERÚ

2025

1. Comparación de las tres representaciones

Se trabajó con tres tipos de representaciones cromosómicas para distribuir a 39 alumnos en tres exámenes de forma equitativa:

Representación	Estructura del cromosoma	Ventaja principal	Desventaja principal
Binaria	117 bits ($39 \text{ alumnos} \times 3 \text{ bits}$), un bit activo por alumno	Simple y directa	Alta penalización si distribución no es válida
Permutacional	Permutación de 39 índices de alumnos	Asegura distribución balanceada	Mayor complejidad en cruce y mutación
Real (probabilística)	117 valores reales normalizados (39×3)	Alta flexibilidad y exploración continua	Requiere normalización y decodificación

2. Resultados y conclusiones de las actividades realizadas

○ Actividad 1: Análisis Comparativo

- Se ejecutaron los tres algoritmos con diferentes representaciones:
- Binaria: alcanzó un equilibrio aceptable entre grupos con una desviación estándar de 0.0363 y se estabilizó alrededor de la generación 20.
- Permutacional: fue la que convergió más rápido, estabilizándose en solo 10 generaciones, con los mismos niveles de equilibrio.
- Real: también logró un equilibrio en promedios (0.0363 de desviación), pero con mayor varianza interna en los grupos y convergencia lenta (sin mejoras notables incluso tras 120 generaciones).

○ Actividad 2: Modificación de Fitness

Se modificó la función calcular_fitness en la representación binaria para:

- Penalizar grupos con alta varianza de notas (menor homogeneidad).
- Premiar diversidad interna, es decir, mezclar alumnos con distintos rendimientos.

Aspecto	Versión Original	Versión Modificada
---------	------------------	--------------------

Fitness final	-0.0363	-1000.0
Restricciones cumplidas	Sí (13 alumnos por grupo)	No (desequilibrio de grupos)
Desviación entre promedios	0.0363	0.8393
Observaciones	Buena convergencia y balance	Alta penalización, sin solución válida

En conclusión, La versión original es más estable y cumple mejor las restricciones del problema.

○ *Actividad 3: Nuevo Operador Genético*

Se implemento nueva función de mutación para la representación real:

```
def mutacion_gaussiana(cromosoma, sigma=0.1):
    cromosoma_mutado = cromosoma.copy()
    for i in range(39):
        idx = i * 3
        genes = cromosoma_mutado[idx:idx+3]
        perturbacion = np.random.normal(0, sigma, 3)
        nuevos_genes = genes + perturbacion
        nuevos_genes = np.clip(nuevos_genes, 0, None)
        suma = np.sum(nuevos_genes)
        if suma == 0:
            nuevos_genes = np.array([1/3, 1/3, 1/3])
        else:
            nuevos_genes = nuevos_genes / suma
        cromosoma_mutado[idx:idx+3] = nuevos_genes
    return cromosoma_mutado
```

Resultados comparativos:

Parámetro (σ)	Fitness Final	Desviación entre Promedios	Varianza A / B / C	Observación
$\sigma = 0.1$	-1.0911	0.0363	9.01 / 10.39 / 12.25	Mutación suave, solución estable
$\sigma = 0.3$	-1.0911	0.0363	15.62 / 8.39 / 7.63	Más diversidad, pero sin pérdida de equilibrio

En conclusión, el uso de mutación gaussiana permite ajustar el nivel de exploración en el espacio de soluciones y una sigma bajo mantiene estabilidad y equilibrio en las soluciones.

○ *Actividad 4: Restricciones Adicionales*

- Se agregó una nueva restricción al programa `representacion_permutacional.py`:
- Regla: los alumnos con nota menor a 11 no pueden estar todos en el mismo examen.
- Para ello, se modificó la función de fitness para penalizar soluciones que violen esta condición, agregando una verificación de distribución de alumnos con bajo rendimiento ($\text{nota} < 11$) entre los grupos.

Versión	Fitness Inicial	Fitness Final	Mejora Total	Desviación entre Promedios
Sin restricción	0.0385	0.2637	584.8%	0.0363
Con restricción aplicada	0.1693	0.2637	55.8%	0.0363

○ **Actividad 5: Visualización (Avanzado)**

Visualización: Se generaron gráficos usando Matplotlib y Seaborn para ilustrar:

- Evolución del fitness por generaciones
- Histogramas de notas por examen
- Comparación visual de los promedios por examen entre representaciones

○ **Actividad 6 (Problema Extendido)**

Se adaptó la representación binaria para trabajar con cuatro exámenes. Se modificó el cromosoma a 156 bits (39×4), y se ajustó la lógica de asignación y validación. La convergencia fue más lenta, debido al aumento del espacio de búsqueda.

3. Reflexión sobre cuándo y por qué usar cada tipo de representación.

- Binaria: Ideal para problemas discretos donde hay asignaciones exclusivas y reglas claras. Su simplicidad permite implementación rápida, pero requiere control estricto para evitar soluciones inválidas.
- Permutacional: Útil cuando se necesita preservar orden o cuando se requiere equilibrio estricto entre grupos, como en scheduling o balanceo de carga. Menos intuitiva de manipular, pero muy efectiva para problemas combinatorios.
- Real (probabilística): Recomendable para problemas continuos o donde se quiera favorecer la exploración. Es más robusta frente a pequeños cambios y admite ajustes finos, pero puede complicar la decodificación.